



XUNTA
DE GALICIA

CONSELLERÍA DE CULTURA,
EDUCACIÓN, FORMACIÓN
PROFESIONAL E UNIVERSIDADES



IES SAN MAMEDE

📍 Rúa do Castelo N° 3. 32700. Maceda (Ourense)
☎ Tlf: 988 788 561 ✉ ies.san.mamede@edu.xunta.gal
🌐 <http://www.iesanmamede.com>

GuzmanGes

<https://github.com/lago-3004/guzmanges-proyecto>

Nome Alumno/a: Iago Malvido Guzmán

Curso: 2º DAM

Módulo: *Proxecto Final Ciclo*

Contido

Contido	2
1. Introducción	3
2. Obxectivos	3
3. Situación previa	3
4. Tecnoloxías empregadas	4
5. Solución proposta	4
6. Planificación do proxecto	5
6.1 Presuposto completo (Hardware, software e man de obra)	5
6.2 Diagrama de Gantt	7
6.3 Modelo de desenvolvemento de software a implementar	7
6.4 Análise do proxecto	8
6.4.1 Definición do alcance do proxecto	8
6.4.2 Universo de discurso da base de datos	11
6.4.3 Diagramas Entidad-Relación	17
6.4.4 Diagrama de casos de uso	17
6.4.5 Diagrama de clases	18
7. Desenvolvemento e execución	18
8. Conclusións e reflexións	18
9. Bibliografía e Webgrafía	18
10. Anexo I – Manual técnico de instalación ou posta en marcha	19
11. Anexo II – Documentación de uso (manuais usuario)	19
12. Anexo III – Outra documentación	19

1. Introducción

O presente proxecto consiste no desenvolvemento dunha aplicación móbil de preventa deseñada para optimizar a xestión comercial e o proceso de toma de pedidos por parte dos representantes de vendas (preventas). A aplicación permite aos usuarios realizar pedidos de forma eficiente directamente desde os dispositivos móbiles, consultando en tempo real os datos dos clientes existentes na base de datos, o catálogo de produtos e os seus prezos. Tamén incorpora a funcionalidade de dar de alta novos clientes.

A aplicación céntrase en reducir os tempos de xestión, minimizar erros nos pedidos e ofrecer unha ferramenta dixital completa e accesible para mellorar a produtividade.

2. Obxectivos

O desenvolvemento deste proxecto persegue un conxunto de metas ben definidas, tanto a nivel xeral de solución de negocio como a nivel específico de funcionalidades técnicas:

Obxectivo xeral:

- **Optimizar e Dixitalizar o Proceso de Preventa:** Desenvolver unha aplicación móbil que substitúa a xestión manual de pedidos por un proceso dixital eficiente, integrado e en tempo real, mellorando a produtividade do equipo comercial.

Obxectivos específicos:

- **Funcionalidade de Pedidos:** Permitir a creación, modificación e envío de pedidos de forma dixital e segura desde a aplicación móbil, cun tempo medio de procesamento inferior a 5 minutos por pedido.
- **Sincronización en Tempo Real:** Garantir a consulta de datos de clientes, produtos e prezos co backend cun tempo de resposta inferior a 2 segundos (cando haxa conexión de datos).
- **Xestión de Clientes:** Implementar a funcionalidade de alta de novos clientes de forma inmediata, validando os datos esenciais no momento da visita comercial.
- **Traballo Sen Conexión (Offline):** Desenvolver un mecanismo de caché local que permita aos preventas realizar pedidos e consultar datos esenciais mesmo sen conexión a Internet, sincronizando a información automaticamente ao recuperar a conectividade.
- **Usabilidade:** Proporcionar unha interface de usuario (UI) intuitiva e optimizada para o traballo en campo, cunha curva de aprendizaxe mínima para o equipo comercial.

3. Situación previa

Actualmente, o proceso de xestión de pedidos por parte dos representantes de vendas da empresa caracterízase pola súa dependencia de métodos manuais e desactualizados. Os preventas adoitan tomar notas en papel ou utilizar follas de cálculo non conectadas, o que leva a varios problemas operativos:

1. **Ineficiencia e Retrasos:** A necesidade de transcribir os pedidos posteriormente ao sistema central (ERP ou sistema de xestión de almacén) provoca atrasos na tramitación e entrega dos produtos.
2. **Erros na Captura de Datos:** A transcripción manual é propensa a erros, como referencias incorrectas de produtos, prezos desactualizados ou condicións comerciais erróneas, o que xera problemas na facturación e insatisfacción do cliente.
3. **Falta de Información en Tempo Real:** Os preventas non dispoñen de acceso inmediato ao stock actual, ás condicións comerciais específicas do cliente ou á súa historia de pedidos, limitando a súa capacidade de venda consultiva e de resposta rápida.

4. **Dificultade na Xestión de Clientes:** O proceso de alta de novos clientes é lento e require coordinación posterior coa oficina.

Existen alternativas no mercado, principalmente sistemas de Xestión de Relacións con Clientes (CRM) ou Módulos de Ventas de ERPs máis amplos (como SAP ou solucións locais específicas). Non obstante, estas alternativas adoitan ser onerosas na súa implantación, requiren unha curva de aprendizaxe elevada e, en moitos casos, ofrecen unha funcionalidade excesivamente ampla para o obxectivo específico dunha preventa simplificada e enfocada na mobilidade.

O presente proxecto busca ofrecer unha solución áxil, económica e especificamente adaptada aos procesos de preventa no contexto da empresa, sen a complexidade nin o custo dun ERP completo.

4. Tecnoloxías empregadas

Para o desenvolvemento do proxecto, empregárase unha arquitectura moderna e orientada a servizos para garantir a escalabilidade e o rendemento, cos seguintes compoñentes clave:

Frontend:

- **Framework: Flutter** (con **Dart**) empregárase para o desenvolvemento da aplicación móbil, o que permitirá crear unha única base de código para despregamento en múltiples plataformas (Android e/ou iOS).

Backend:

- **Framework:** Utilizarase **Spring Boot** para desenvolver a API RESTful que actuará como o núcleo da lóxica de negocio. Esta API encargárase de xestionar as peticións do frontend, a autenticación e a interacción coa base de datos.
- **Base de datos:** Os datos estruturados do sistema (clientes, produtos, pedidos) almacenaranse nunha base de datos relacional MySQL. Na aplicación móbil, poderase implementar un sistema de caché local para almacenar datos esenciais e permitir o traballo sen conexión.

5. Solución proposta

A solución proposta consiste no desenvolvemento de GuzmanGes, unha aplicación móbil nativa multiplataforma (grazas a Flutter) xunto cunha robusta API RESTful (desenvolvida con Spring Boot) e unha base de datos MySQL centralizada.

Esta arquitectura separa claramente a lóxica de negocio (no backend Spring Boot) da interface de usuario (frontend Flutter), garantindo a escalabilidade, o rendemento e un mantemento sinxelo.

Características diferenciais e valor engadido:

- **Deseño Específico para Preventa:** A diferenza dos módulos de vendas xenéricos de ERPs, GuzmanGes está deseñada exclusivamente para simplificar o proceso de toma de pedidos en campo. A interface e os fluxos de traballo priorizan a velocidade e a mínima interacción.
- **Funcionalidade Offline (Traballo Sen Conexión):** Esta é unha vantaxe clave fronte a moitas solucións baseadas unicamente na web ou en conexión permanente. O sistema de caché local con SQLite na aplicación móbil permite traballar sen conexión, garantindo a continuidade do negocio mesmo en áreas con baixa cobertura de rede, para sincronizar os pedidos tan pronto como se recupere a conectividade.
- **Tecnoloxía Moderna e Eficiente:** A elección de Flutter permite un desenvolvemento rápido e despregamento en Android e iOS cunha única base de código. Spring Boot, coñecido pola súa eficiencia na creación de microservizos e APIs, asegura a dispoñibilidade e rapidez do backend.

- **Integración Sencilla:** A través da API RESTful, o sistema está deseñado para unha futura integración sinxela con outros sistemas internos da empresa (como o sistema de xestión de almacén ou a contabilidade), minimizando o rozamento na adopción.

6. Planificación do proxecto

6.1 Presuposto completo (Hardware, software e man de obra)

A seguinte táboa presenta unha estimación de custos revisada para o desenvolvemento completo do proxecto, asumindo unha duración de 3 meses e unha dedicación de 40 horas semanais, cunhas tarifas de man de obra axustadas a un custo máis económico.

1. Recursos Humanos (Man de Obra)

Rol	Dedicación (Horas)	Custo Hora Estimado (€)	Custo Total (€)
Desenvolvedor Senior/Arquitecto	480	15,00 €	7.200,00 €
Desenvolvedor Junior/Móbil	480	10,00 €	4.800,00 €
Xestor de Proxectos/Análise (25% tempo)	120	20,00 €	2.400,00 €
Subtotal Recursos Humanos	1080		14.400,00 €

2. Software e Licenzas

Para o desenvolvemento, optouse por tecnoloxías de código aberto e gratuítas que minimizan os custos de licenza, excepto o custo do aloxamento.

Descrición	Tipo de Custo	Custo Estimado (€)
Flutter/Dart/Spring Boot/MySQL	Licenza (Open Source)	0,00 €
IDE's de Desenvolvemento (Versións gratuítas)	Licenza	0,00 €
Servidor de Repositorio (GitHub/GitLab)	Servizo (Versións gratuítas)	0,00 €

Servidor de Despregamento (VPS/Cloud)	Custo de subscrición/mes (Estimación)	450,00 €
Subtotal Software e Servizos		450,00 €

3. Hardware

O hardware de desenvolvemento (portátiles, dispositivos de proba) asúmese como xa existente e amortizado pola empresa.

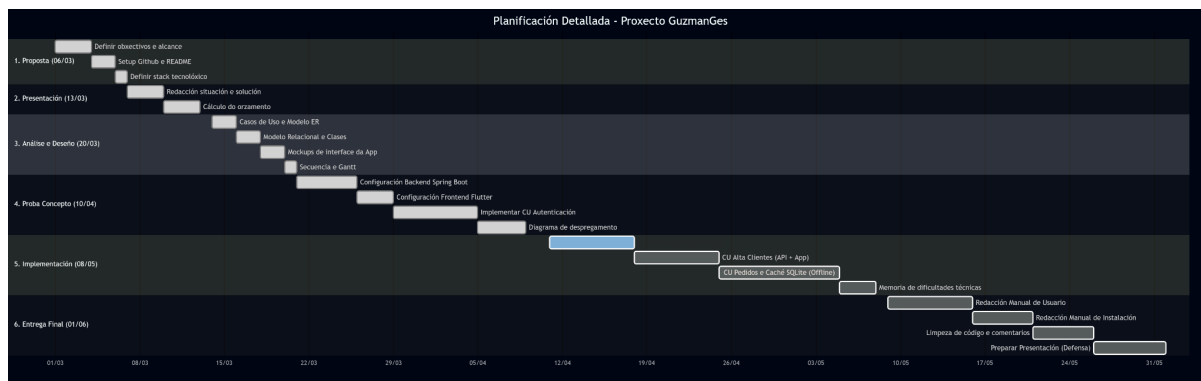
Descrición	Tipo de Custo	Custo Estimado (€)
Equipos de Desenvolvemento e Proba	Custo xa asumido/amortizado	0,00 €
Tablet/Móbil para Empregados (e.g. Samsung/equivalente)	Custo de adquisición/amortización	200,00 €
Subtotal Hardware		200,00 €

4. Orzamento Total do Proxecto

Concepto	Custo Total (€)
Recursos Humanos	14.400,00 €
Software e Servizos	450,00 €
Hardware	200,00 €
TOTAL ORZAMENTO ESTIMADO	15.050,00 €

Nota: Este orzamento é unha estimación baseada en tarifas axustadas. O custo real do servidor de despregamento dependerá do provedor e da carga de traballo esperada. Os prezos indicados anteriormente son sin impostos incluídos.

6.2 Diagrama de Gantt



6.3 Modelo de desenvolvemento de software a implementar

Para o desenvolvemento de GuzmanGes, optouse por unha metodoloxía de desenvolvemento Áxil baseada no marco de traballo Scrum. Esta elección responde á necesidade dun proceso iterativo e incremental que permita a evolución constante tanto da aplicación móbil (Flutter) como da API RESTful (Spring Boot)

1. Xustificación do modelo:

- Desenvolvemento Iterativo e Incremental:** Dado que o proxecto require a integración de múltiples compoñentes (Frontend, Backend e Base de Datos), Scrum permite realizar entregas parciais e funcionais en períodos curtos.
- Xestión de Riscos Técnicos:** A funcionalidade de traballo sen conexión (offline) e o uso de caché local con SQLite son puntos críticos que requiren ciclos frecuentes de proba e axuste, algo que este modelo facilita enormemente.
- Foco na Usabilidade:** Ao ser unha ferramenta para representantes de vendas en campo, a interface debe ser altamente intuitiva e cunha curva de aprendizaxe mínima. O enfoque áxil permite adaptar o deseño de Flutter baseándose no feedback continuo a medida que se constrúen as pantallas.

2. Implementación do proceso (Sprints)

O desenvolvemento estrutúrase en ciclos de traballo denominados **Sprints**, os cales se aliñaron cos fitos de entrega establecidos no calendario oficial do proxecto:

- Sprint 1 (Análise e Deseño):** Definición dos requisitos, deseño do modelo Entidade-Relación, diagrama de casos de uso e creación de *mockups* da interface.
- Sprint 2 (Proba de Concepto):** Configuración das contornas de desenvolvemento (Spring Boot e Flutter), diagrama de despliegamento e implementación do primeiro caso de uso (habitualmente a Autenticación).
- Sprint 3 (Implementación de Funcionalidades):** Desenvolvemento do resto do alcance acordado: catálogo de produtos, xestión de clientes, creación de pedidos e a lóxica de sincronización bidireccional *offline/online*.
- Sprint 4 (Peche e Documentación):** Refinado do código, elaboración da memoria de dificultades técnicas, redacción dos manuais (usuario e instalación) e preparación da presentación final.

6.4 Análise do proxecto

6.4.1 Definición do alcance do proxecto

1.- Visión xeral

GuzmanGes é unha aplicación móbil multiplataforma (Android e iOS, desenvolvida con Flutter) destinada aos representantes comerciais (preventas) que visitan clientes en campo. A aplicación permítelles consultar o catálogo de produtos, a cartera de clientes e o histórico de pedidos sincronizados desde o ERP corporativo **Odoo**, así como rexistrar novos pedidos e dar de alta clientes novos durante a propia visita.

A aplicación apóiase nunha **API REST en Spring Boot** que actúa como capa intermedia entre Odoo e o dispositivo móbil, e nunha base de datos **MySQL** que almacena os datos sincronizados desde Odoo e os pedidos novos xerados desde a app.

2. Que fai o proxecto (dentro do alcance)

2.1. Backend (API REST en Spring Boot + MySQL)

- **Autenticación** de usuarios mediante login/contrasinal con tokens JWT.
- **Sincronización con Odoo** (capa Spring Boot ↔ Odoo vía XML-RPC ou JSON-RPC):
 - Importación periódica desde Odoo a MySQL de:
 - Clientes (res.partner).
 - Produtos (product.product).
 - Modos de pagamento e condicións de pagamento.
 - Pedidos do **último ano** (sale.order + sale.order.line).
 - Exportación cara Odoo dos pedidos novos creados desde a aplicación móbil.
 - Exportación cara Odoo dos clientes novos dados de alta desde a aplicación.
- **Endpoints REST** consumidos pola aplicación móbil:
 - Autenticación.
 - Lectura de clientes, produtos, modos e condicións de pagamento, e histórico de pedidos.
 - Creación, modificación, eliminación e envío de pedidos.
 - Creación de clientes novos.
- **Persistencia en MySQL** de toda a información sincronizada e dos novos rexistros xerados desde a app, incluídos os identificadores idOdoo para reconciliación.

2.2. Aplicación móbil (Flutter)

A aplicación está deseñada como un **cliente principalmente de lectura** sobre os datos mestres (clientes, produtos, catálogos), agás polas operacións explícitas que se detallan a continuación.

Operacións de lectura (consulta de datos sincronizados desde Odoo):

- Listaxe e busca de clientes.
- Visualización do detalle dun cliente (datos mestres + histórico de pedidos do último ano, sexan creados na app ou en Odoo).
- Listaxe e busca de produtos.
- Visualización do detalle dun produto.
- Consulta de modos e condicións de pagamento.

Operacións de escritura permitidas dende a app:

- **Alta de cliente** novo con validación dos datos básicos obrigatorios (CIF, razón social, dirección, localidade, código postal, provincia, teléfono).
 - Non se permite editar nin eliminar clientes existentes desde a app: a edición faise en Odoo.
- **CRUD completo de pedidos** mentres estean en estado borrador
 - Crear pedido (cabeceira + liñas con produto, cantidade, prezo, IVE, recargo de equivalencia).
 - Editar pedido borrador (engadir, modificar ou eliminar liñas).
 - Eliminar pedido borrador.
 - **Confirmar/enviar pedido:** cambia o estado a confirmado e queda bloqueado para edición. A partir dese momento o sistema o sincroniza con Odoo.

2.3. Sincronización Flutter ↔ Spring Boot (caché local SQLite)

- A aplicación móbil mantén unha **caché local en SQLite** cos datos mestres (clientes, produtos, catálogos) descargados na última conexión, para permitir o traballo en zonas sen cobertura.
- **Cola de envío** local en SQLite para pedidos creados ou modificados sen conexión, e para clientes novos dados de alta sen conexión.
- Ao recuperar conexión, a app envía a cola á API REST en orde cronolóxica.
- A sincronización é **unidireccional na escritura** (do móbil cara o servidor): non se contempla edición concorrente offline de rexistros existentes.

3. Que NON fai o proxecto (fóra do alcance)

Quedan explicitamente fóra do alcance do MVP as seguintes funcionalidades:

- **Edición ou eliminación de clientes existentes** desde a app móbil (xestiónase en Odoo).
- **Xestión do catálogo de produtos** desde a app (alta, edición, baixa, prezos, stock — todo se xestiona en Odoo).
- **Xestión de modos e condicións de pagamento** desde a app.
- **Edición de pedidos xa confirmados ou xa exportados a Odoo** desde a app (só se permite editar pedidos en estado borrador).
- **Resolución de conflitos en escritura concorrente offline:** non se contempla que dous comerciais editen o mesmo rexistro sen conexión simultaneamente.
- **Xestión de usuarios e permisos** desde a app (alta, baixa, cambio de contrasinal — xestiónanse no backend ou directamente en BD).
- **Xeolocalización** dos comerciais ou rutas de visitas.
- **Xeración de reportes**, estatísticas, gráficas ou cadros de mando.
- **Xestión de pagamentos** ou cobros desde a app.
- **Soporte multi-empresa ou multi-divisa.**
- **Notificacións push** (avisos de novos clientes, alertas de stock, etc.).
- **Integración con sistemas externos distintos de Odoo** (CRMs, plataformas de e-commerce, contabilidade externa, etc.).

4. Sistemas externos

4.1. Odoo (ERP)

Aspecto	Descrición
Forma parte do proxecto?	Si , a integración con Odoo está dentro do alcance.
Rol	Sistema de información mestre para clientes, produtos, catálogos de pagamento e pedidos da empresa.
Versión asumida	Odoo Community Edition (autoinstalado en Docker)
Protocolo de integración	XML-RPC ou JSON-RPC (API estándar de Odoo). A elección concreta documentarase na memoria de execución.
Modelos de Odoo implicados	res.partner (clientes), product.product (produtos), account.payment.term (condicións de pagamento), sale.order e sale.order.line (pedidos).
Dirección da sincronización	Bidireccional: Odoo → MySQL (lectura periódica de mestres e pedidos do último ano) e MySQL → Odoo (envío de pedidos novos e clientes novos).
Reconciliación	Cada entidade sincronizable na BD ten un campo idOdoo que almacena o identificador en Odoo. Os rexistros creados desde a app reciben o seu idOdoo despois da subida exitosa a Odoo.
Frecuencia de sync	Sincronización dende Odoo programada periodicamente (xob no backend) ou disparada manualmente desde a app mediante o botón "actualizar". A subida de pedidos faise inmediatamente tras a confirmación cando hai conexión, ou desde a cola de SQLite ao recuperala.

4.2. Outros sistemas

- **Servidor de despregamento (VPS / Cloud):** infraestrutura externa onde se hospeda a API Spring Boot e a base de datos MySQL. Non forma parte do desenvolvemento, pero si do despregamento.
- **Repositorio de código (GitHub):** servizo externo para o control de versións. Non forma parte do desenvolvemento funcional.

5. Restricións e supostos

Restricións técnicas

- A aplicación móbil debe funcionar en **Android e iOS** desde unha única base de código (Flutter).

- O backend está restrinxido ao stack acordado: **Spring Boot 4 + Java 21 + MySQL**.
- O tempo de resposta dos endpoints de lectura debe ser inferior a **2 segundos** cando hai conexión.
- O tempo medio de procesamento dun pedido completo (creación + confirmación) debe ser inferior a **5 minutos**.

Supostos de negocio

- Cada comercial ten asignada unha cartera de clientes (relación CLIENTE.comercial_id).
- O catálogo de produtos é común a toda a empresa.
- Os prezos de venda son os definidos en Odoo. Non se contempla negociación de prezos no momento da venda desde a app (o prezo de cada liña é o do produto en Odoo no momento da última sincronización).
- O comercial nunca elimina rexistros de Odoo desde a app: as eliminacións (clientes, produtos) realízanse exclusivamente en Odoo.

Supostos de despregamento

- Existe unha instancia de Odoo Community Edition accesible desde o backend Spring Boot.
- O backend está hospedado nun servidor con conectividade permanente a internet.
- O dispositivo móbil ten conectividade intermitente (4G/5G/WiFi) e debe poder operar nas xanelas sen conexión.

6.4.2 Universo de discurso da base de datos

1. Contexto

A empresa para a que se desenvolve **GuzmanGes** dedícase á distribución de produtos e mantén un equipo de **representantes comerciais (preventas)** que visitan clientes en campo para tomar pedidos. Toda a información de clientes, produtos e pedidos histórica vive no sistema de planificación de recursos da empresa, **Odoo**, que actúa como sistema mestre.

A base de datos relacional **MySQL** que sustenta a API Spring Boot replica a información necesaria de Odoo e almacena os pedidos novos xerados desde a aplicación móbil mentres non son enviados a Odoo, así como os pedidos confirmados xa sincronizados.

2. Descrición das entidades

2.1. Usuario

A empresa dispón dun **conxunto de usuarios** que poden iniciar sesión na aplicación móbil. Cada usuario representa un comercial (preventas) ou un perfil administrativo. De cada usuario almacénanse os seus datos persoais e de acceso: nome completo, nome de usuario único para o login, enderezo de correo electrónico, contrasinal cifrado e un tipo de usuario que identifica o seu rol (por exemplo, comercial ou administrador).

Os usuarios non se sincronizan con Odoo: créanse e mantéñense exclusivamente no sistema GuzmanGes.

2.2. Cliente

Un **cliente** representa un local, comercio ou empresa á que se lle venden produtos. De cada cliente almacénanse:

- Identificador interno (xerado automaticamente).
- Identificador en Odoo (idOdoo): permite reconciliar o rexistro co sistema mestre.
- Nome comercial e razón social.
- CIF (identificación fiscal).
- Dirección postal completa: rúa, localidade, código postal e provincia.
- Datos de contacto: teléfono fixo, móbil, correo electrónico.
- Posición fiscal (réxime fiscal aplicable: por exemplo, réxime xeral, recargo de equivalencia).
- Marca de actividade (activo) e data da última modificación.

Cada cliente ten asignados de xeito obrigatorio:

- Un **modo de pagamento** (forma na que paga: transferencia, tarxeta, contado, etc.).
- Unha **condición de pagamento** (prazos: contado, 30 días, 60 días, 90 días, etc.).
- Un **comercial** responsable do cliente, é dicir, o usuario que xestiona habitualmente esa conta.

Os clientes provéñen maioritariamente de Odoo (sincronización descendente), aínda que o sistema permite dar de alta clientes novos desde a aplicación móbil. Eses novos clientes envíanse a Odoo na seguinte sincronización ascendente, momento no que reciben o seu idOdoo definitivo.

2.3. Modo de pagamento

Catálogo simple xestionado en Odoo. Cada **modo de pagamento** ten un identificador interno, un identificador en Odoo (idOdoo), unha descrición textual (por exemplo, transferencia bancaria, efectivo, cheque), unha marca de actividade e unha data da última modificación. Sincronízase periodicamente desde Odoo a MySQL e é só de lectura na aplicación móbil.

2.4. Condición de pagamento

Catálogo simple xestionado en Odoo. Cada **condición de pagamento** ten un identificador interno, un identificador en Odoo (idOdoo), unha descrición textual (por exemplo, contado, 30 días, 60 días fin de mes), unha marca de actividade e unha data da última modificación. Sincronízase periodicamente desde Odoo a MySQL e é só de lectura na aplicación móbil.

2.5. Produto

Un **produto** representa un ítem do catálogo de venda da empresa. De cada produto almacénanse:

- Identificador interno e identificador en Odoo (idOdoo).
- Referencia interna do produto (código curto usado pola empresa).
- Código de barras.
- Descrición.
- Tipo de produto (por exemplo, almacenable, consumible, servizo).
- Stock dispoñible (cantidade en almacén segundo a última sincronización).
- Prezo de venda.
- Observacións ou notas internas.
- Data da última modificación.

O catálogo de produtos é mestre en Odoo: sincronízase periodicamente cara MySQL e é só de lectura na aplicación móbil. Non se xestiona alta, modificación nin baixa de produtos desde GuzmanGes.

2.6. Pedido

Un **pedido** é unha orde de compra realizada por un cliente nunha visita do comercial (ou ben rexistrada directamente en Odoo desde a oficina). De cada pedido almacénanse:

- Identificador interno e identificador en Odoo (idOdoo, que estará baleiro mentres o pedido aínda non se enviou a Odoo).
- Número de pedido (referencia visible).
- Data de creación.
- Importes calculados: base imponible (totalBase), total de IVE (totalIva), total de recargo de equivalencia (totalRE) e total final (total).
- **Estado do pedido:** describe a fase comercial do pedido. Os valores posibles son:
 - borrador: pedido en construción, modificable.
 - confirmado: pedido pechado polo comercial, listo para enviar a Odoo.
 - anulado: pedido cancelado.
- **Estado de sincronización:** describe se o pedido foi enviado correctamente a Odoo. Os valores posibles son:
 - pendente: aínda non enviado a Odoo.
 - sincronizado: enviado correctamente, ten idOdoo.
 - erro: a subida fallou e debe reintentarse.
- Data da última modificación.

Cada pedido pertence sempre a:

- Un único **cliente**.
- Un único **usuario** (o comercial que rexistrou o pedido). No caso de pedidos importados desde Odoo que non foran feitos por ningún comercial da app, asígnaselles un usuario administrativo predeterminado.

Un pedido contén unha ou varias **liñas de pedido** que detallan os produtos solicitados.

2.7. Liña de pedido

Unha **liña de pedido** representa cada produto incluído nun pedido coa súa cantidade e prezo. De cada liña almacénanse:

- Identificador interno.
- Código do produto (snapshot da referencia no momento do pedido).
- Descrición do produto (snapshot).
- Prezo unitario aplicado.
- Porcentaxe de IVE.
- Porcentaxe de recargo de equivalencia (no caso de clientes con esa posición fiscal).
- Subtotal da liña.

Cada liña pertence a un único **pedido** e referencia un único **produto**. As liñas non existen de xeito independente: cando se elimina un pedido borrador, elimínanse tamén as súas liñas.

Almacénanse o código e a descrición do produto **na propia liña** (en lugar de obter os datos do produto referenciado) para conservar a información histórica do pedido aínda que o produto orixinal se modifique ou se elimine en Odoo posteriormente.

3. Relacións entre entidades

A continuación descríbense as relacións do dominio coas súas cardinalidades.

3.1. Usuario — Cliente (*xestiona*)

Cada **cliente** está asignado a un **comercial** (usuario) que é o responsable habitual da conta. Un mesmo usuario pode xestionar moitos clientes (a súa carteira), e cada cliente ten exactamente un comercial asignado.

- Un Usuario xestiona **0..N** Clientes.
- Un Cliente é xestionado por **1** Usuario.

3.2. Cliente — Modo de pagamento (*paga con*)

Cada **cliente** ten asociado un único **modo de pagamento** habitual. Un mesmo modo de pagamento pode estar asignado a moitos clientes.

- Un Cliente paga con **1** Modo de pagamento.
- Un Modo de pagamento asígnase a **0..N** Clientes.

3.3. Cliente — Condición de pagamento (*condición*)

Cada **cliente** ten asociada unha única **condición de pagamento** habitual. Unha mesma condición pode estar asignada a moitos clientes.

- Un Cliente ten **1** Condición de pagamento.
- Unha Condición de pagamento asígnase a **0..N** Clientes.

3.4. Cliente — Pedido (*realiza*)

Un **cliente** pode ter múltiples pedidos ao longo do tempo. Cada pedido pertence sempre a un único cliente.

- Un Cliente realiza **0..N** Pedidos.
- Un Pedido realízase para **1** Cliente.

3.5. Usuario — Pedido (*rexistra*)

Cada **pedido** foi rexistrado por un único **usuario** (o comercial que o tomou en visita ou un usuario administrativo no caso de pedidos importados desde Odoo). Un usuario pode ter rexistrado moitos pedidos.

- Un Usuario rexistra **0..N** Pedidos.
- Un Pedido é rexistrado por **1** Usuario.

3.6. Pedido — Liña de pedido (*contén*)

Un **pedido** contén unha ou varias **liñas**, e cada liña pertence sempre a un único pedido. As liñas son entidades **débiles**: non existen sen o pedido pai e elimínanse en cascada se o pedido se elimina.

- Un Pedido contén **1..N** Liñas de pedido.

- Unha Liña de pedido pertence a **1** Pedido.

3.7. Produto — Liña de pedido (referencia)

Cada **liña de pedido** referencia un único **produto** do catálogo. Un mesmo produto pode aparecer en moitas liñas de moitos pedidos distintos.

- Un Produto aparece en **0..N** Liñas de pedido.
- Unha Liña de pedido referencia **1** Produto.

4. Restricións e regras de negocio

A continuación recóllense as principais restricións de integridade e regras de negocio que se reflectirán no modelo de datos:

1. **Integridade referencial:** as eliminacións de clientes, produtos, modos e condicións de pagamento están restrinxidas: non se pode eliminar un rexistro mestre se hai pedidos ou clientes que o referencian.
2. **Eliminación en cascada:** ao eliminar un pedido en estado borrador, elimínanse automaticamente as súas liñas.
3. **Inmutabilidade de pedidos confirmados:** un pedido en estado confirmado ou anulado non se pode editar nin eliminar desde a aplicación.
4. **Reconciliación con Odoo:**
 - a. Os rexistros descargados de Odoo teñen sempre idOdoo definido.
 - b. Os rexistros creados desde a aplicación nacen con idOdoo baleiro e enchense unha vez sincronizados con éxito.
 - c. O par (idOdoo, tipo de entidade) é único cando idOdoo non é nulo.
5. **Xanela temporal de sincronización:** desde Odoo só se importan os pedidos do **último ano** respecto á data actual. Os pedidos máis antigos permanecen exclusivamente en Odoo.
6. **Identificadores de usuario:** o nombreUsuario do usuario é único en todo o sistema.
7. **Datos obrigatorios na alta de cliente desde a app:** ao crear un cliente desde a aplicación móbil, son obrigatorios CIF, razón social, dirección, localidade, código postal, provincia, modo de pagamento e condición de pagamento.
8. **Coherencia de totais:** o total dun pedido debe ser igual á suma de subtotais das súas liñas máis os impostos correspondentes (totalBase + totalIva + totalRE). Esta coherencia garántease a lóxica do backend ao gardar o pedido.
9. **Consistencia entre estado e idOdoo:** un pedido con estadoSync = sincronizado debe ter idOdoo definido. Un pedido con estadoSync = pendente ou erro pode ter idOdoo baleiro (se nunca chegou a Odoo) ou definido (se sincronizou nalgún momento e logo se modificou).

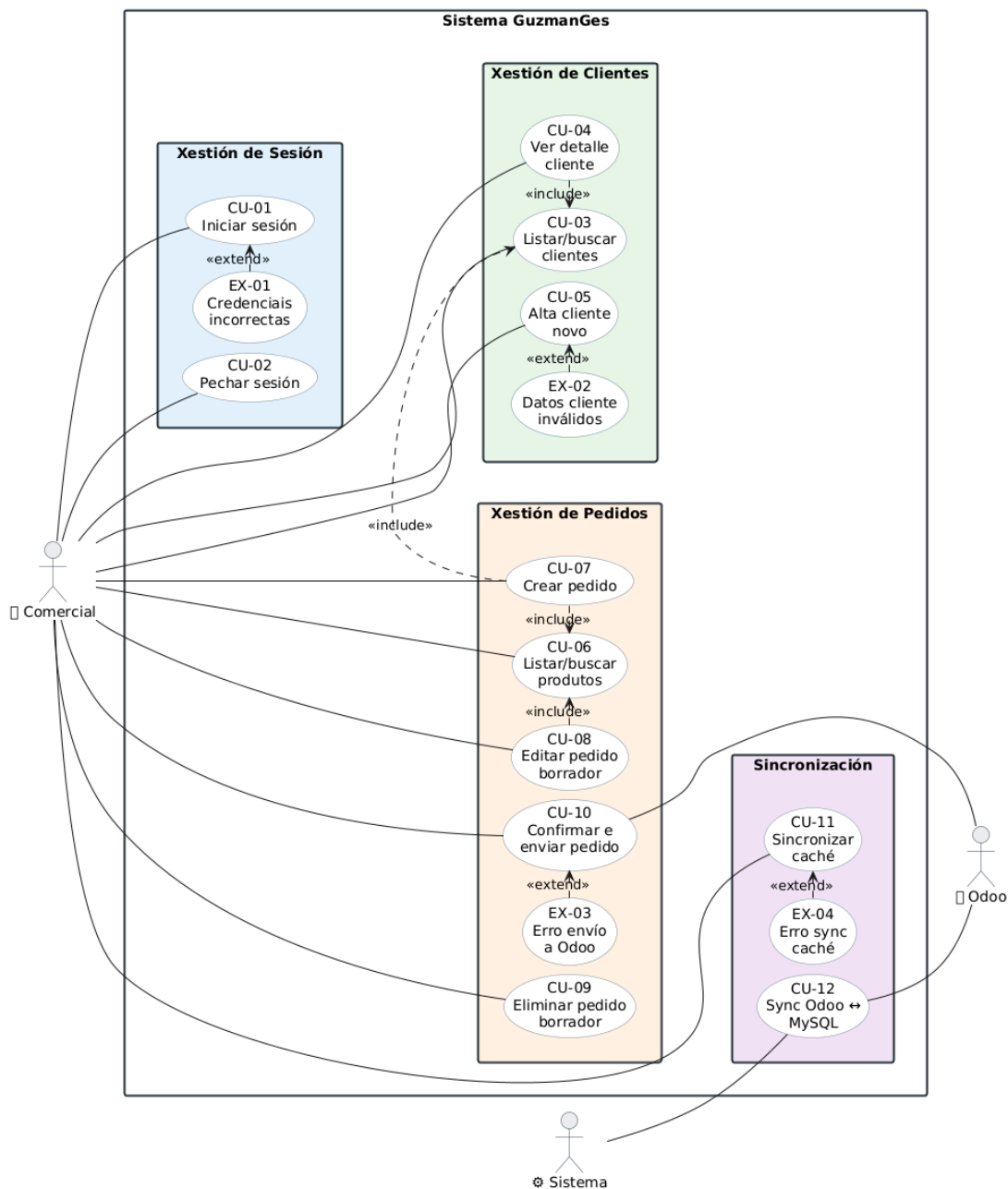
5. Resumo das entidades e cardinalidades

Entidade	Tipo	Sincronizable con Odoo	Editable desde a app
Usuario	Forte	Non	Non (xestionado en BD)
Cliente	Forte	Si (bidireccional)	Só alta
Modo de pagamento	Forte (catálogo)	Si (descendente)	Non
Condición de pagamento	Forte (catálogo)	Si (descendente)	Non
Produto	Forte (catálogo)	Si (descendente)	Non
Pedido	Forte	Si (bidireccional)	CRUD completo en estado borrador
Liña de pedido	Débil (depende de Pedido)	Si (xunto co Pedido)	CRUD coa cabeceira borrador

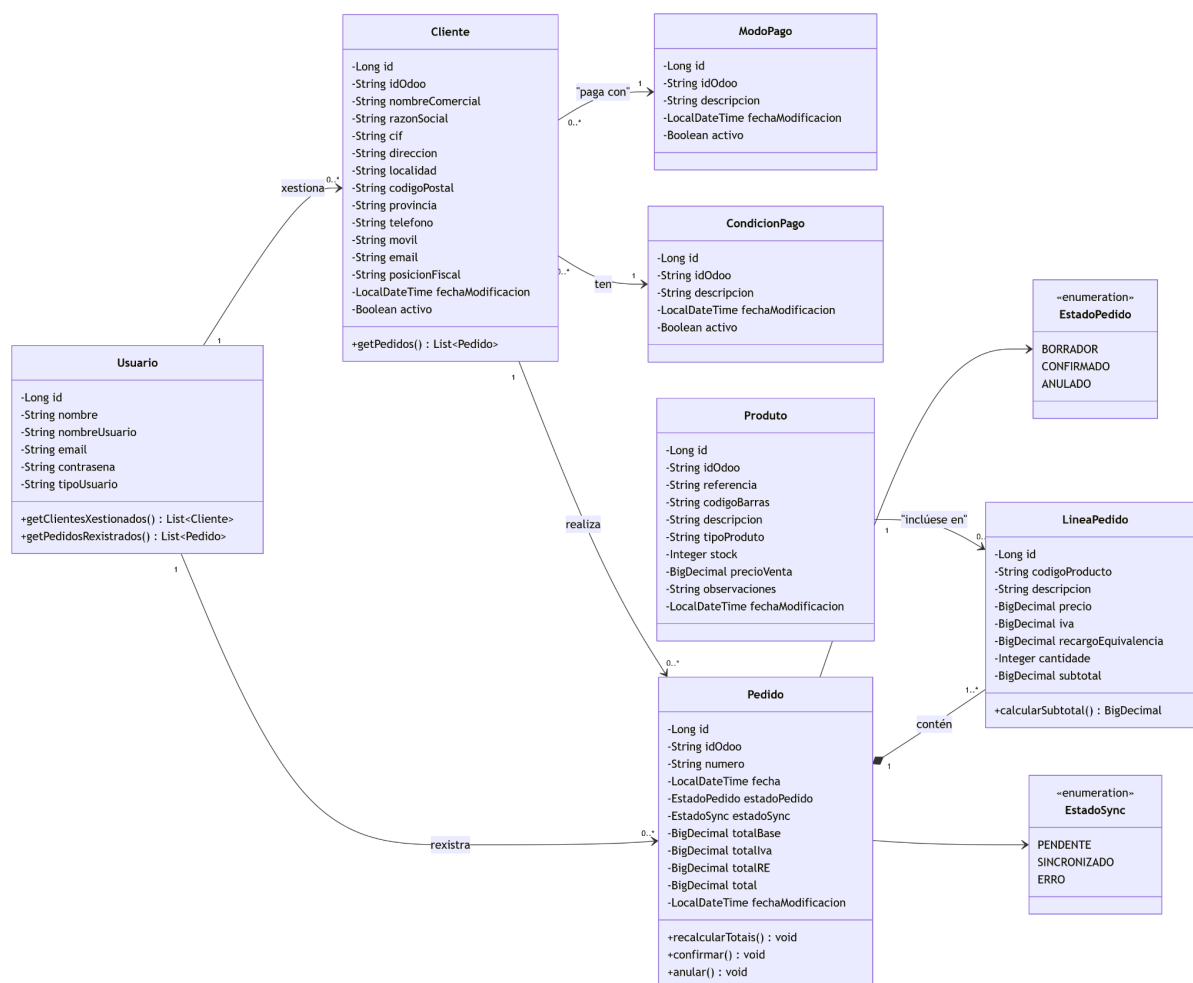
Relación	Cardinalidade
Usuario xestiona Cliente	1 : 0..N
Cliente paga con Modo de pagamento	N : 1
Cliente ten Condición de pagamento	N : 1
Cliente realiza Pedido	1 : 0..N
Usuario rexistra Pedido	1 : 0..N
Pedido contén Liña de pedido	1 : 1..N
Produto referenciado por Liña de pedido	1 : 0..N

6.4.3 Diagramas Entidad-Relación

6.4.4 Diagrama de casos de uso



6.4.5 Diagrama de clases



7. Desenvolvemento e execución

Diagrama de despregamento e descripción xeral do funcionamento da aplicación.

Completar cos detalles técnicos que fose preciso resolver documentados ao longo do desenvolvemento.

8. Conclusións e reflexións

Valoración global do proxecto, resume do aprendido e das dificultades atopadas.

9. Bibliografía e Webgrafía

[Formato APA](#)



10. Anexo I – Manual técnico de instalación ou posta en marcha

11. Anexo II – Documentación de uso (manuais usuario)

12. Anexo III – Outra documentación

Calquera material adicional relevante (documentación da aplicación, manual de uso, capturas de pantalla, fragmentos de código, etc.).